Fake News Detection Project

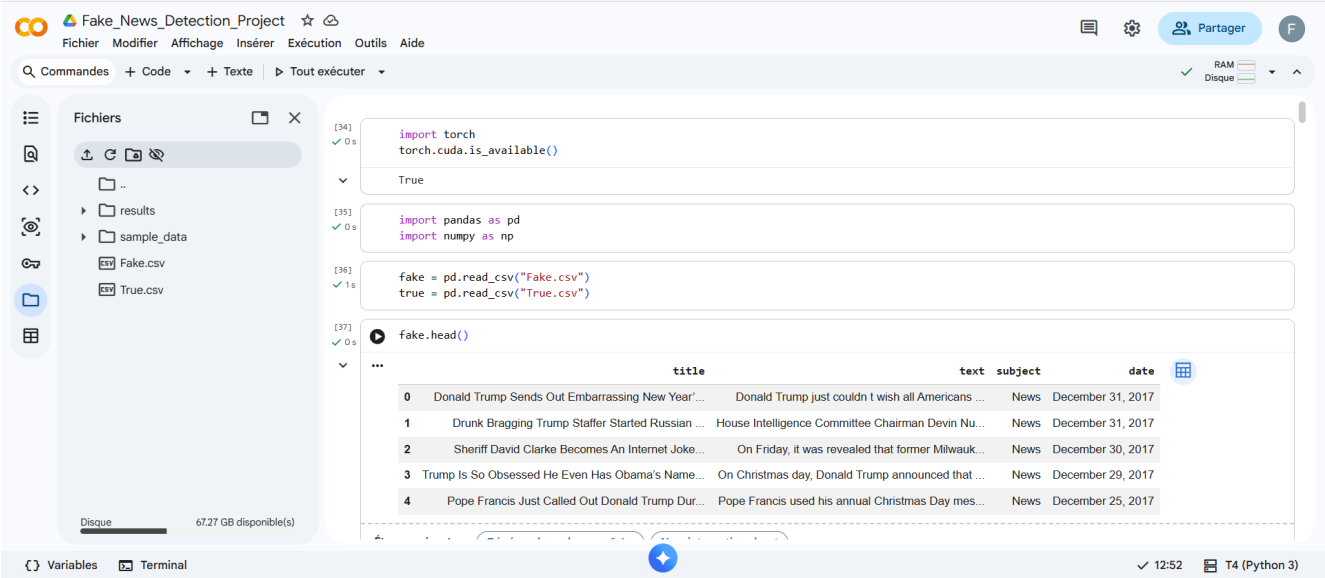
Machine Learning baseline (TF-IDF) and Deep Learning model (DistilBERT)

This report summarizes the complete workflow of the project, from data loading and preprocessing to exploratory data analysis, model training, final evaluation, and performance comparison. The screenshots included in this document were taken directly from the Google Colab notebook used during development.

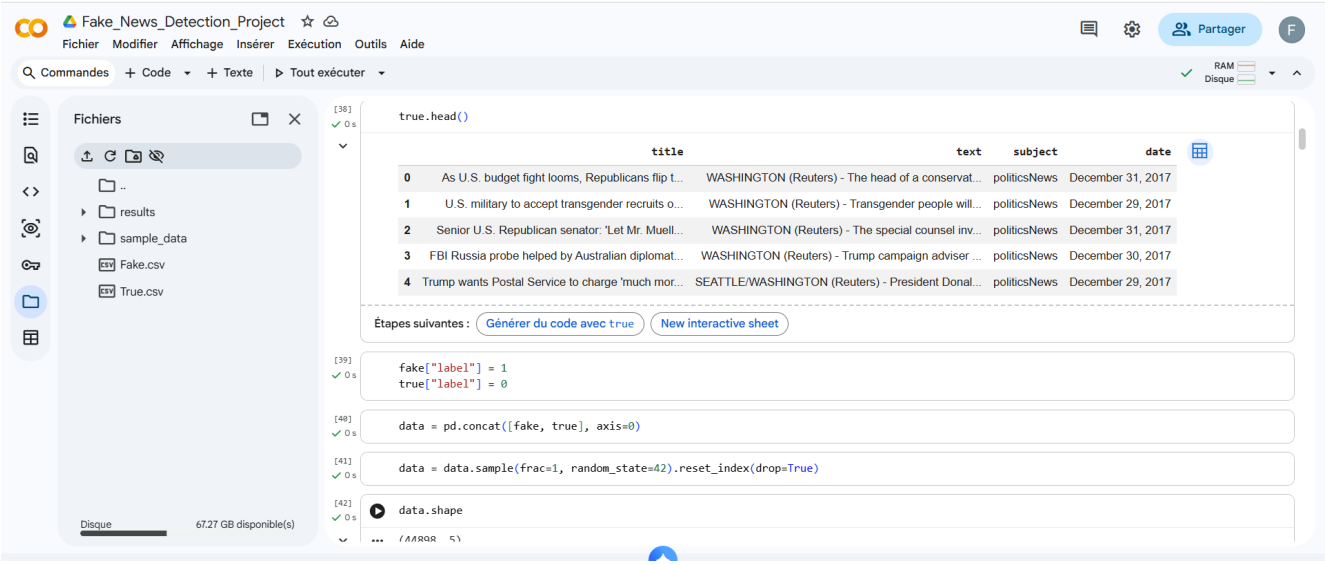
Main steps covered:
- Data loading and inspection
- Preprocessing and train/test split
- Exploratory Data Analysis (EDA)
- TF-IDF + Logistic Regression baseline
- DistilBERT fine-tuning
- Accuracy, classification report, confusion matrix, and model comparison

1. Data loading and initial inspection

The project starts by enabling GPU in Google Colab, importing the main Python libraries, and loading the two CSV files: **Fake.csv** and **True.csv**. The first rows of each dataset are displayed in order to verify the available columns such as title, text, subject, and date. This stage confirms that the raw data is accessible and ready for preprocessing.



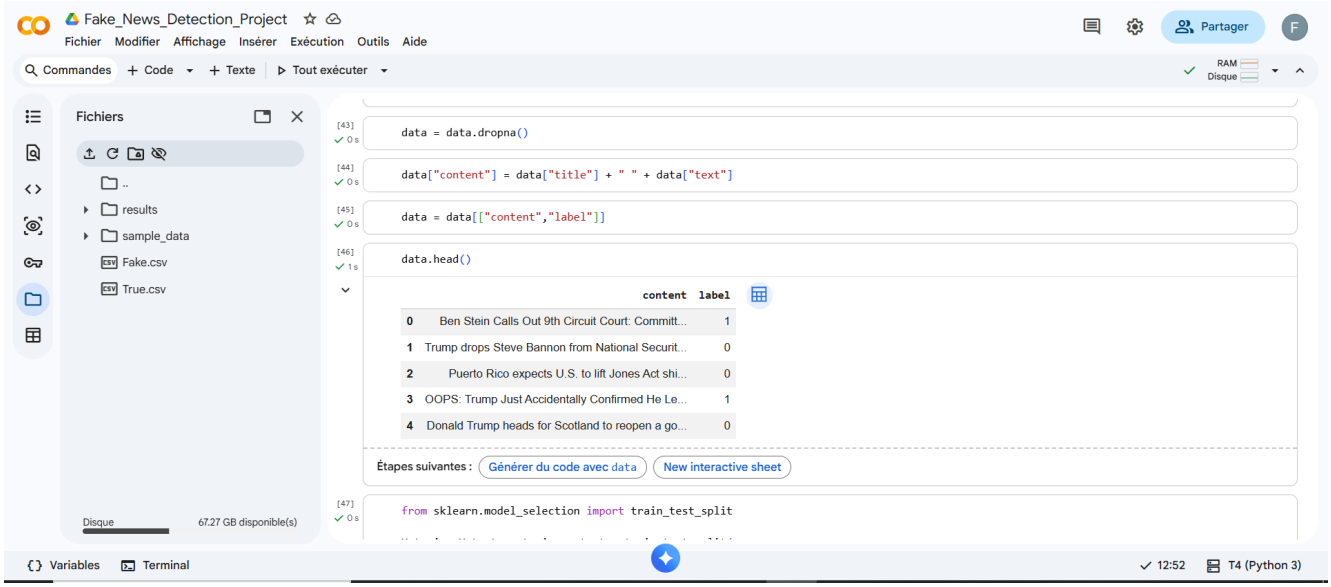
GPU and library setup, followed by loading Fake.csv.



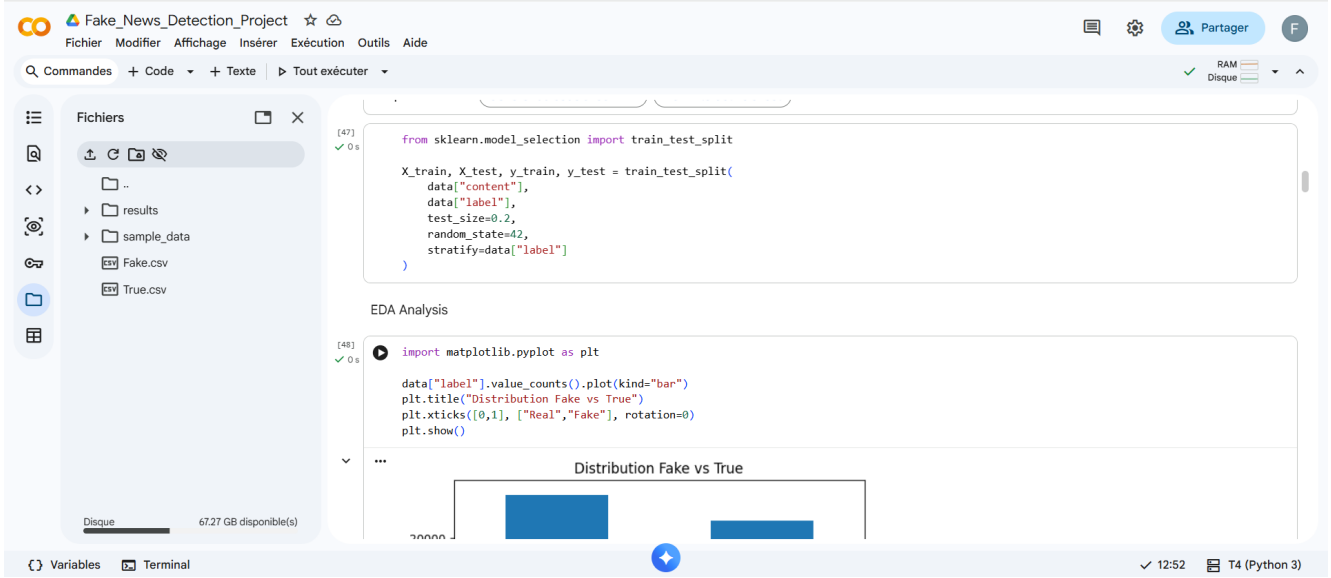
Inspection of the first rows of True.csv and start of dataset merging.

2. Preprocessing and train-test split

During preprocessing, labels are assigned to each class (**Fake = 1** and **Real = 0**), the two datasets are concatenated, shuffled, cleaned from missing values, and a new **content** column is created by combining the article title with the article body. Finally, the data is split into training and testing subsets using **train_test_split** while preserving class balance through stratification.



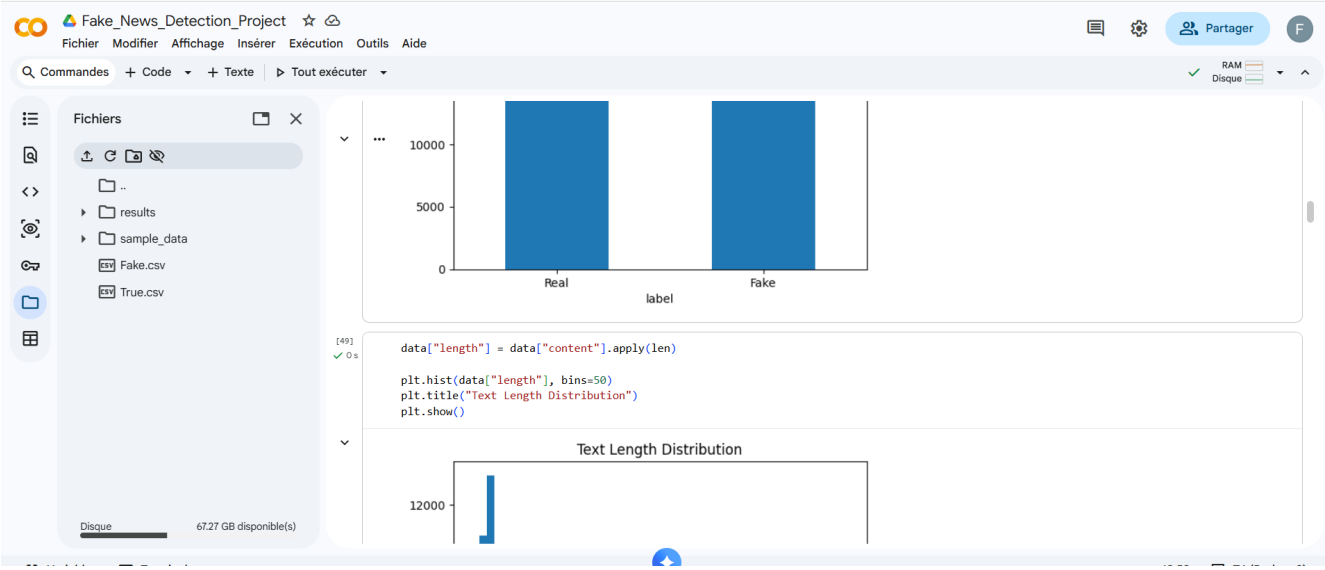
Creation of labels, concatenation, cleaning, and generation of the combined content field.



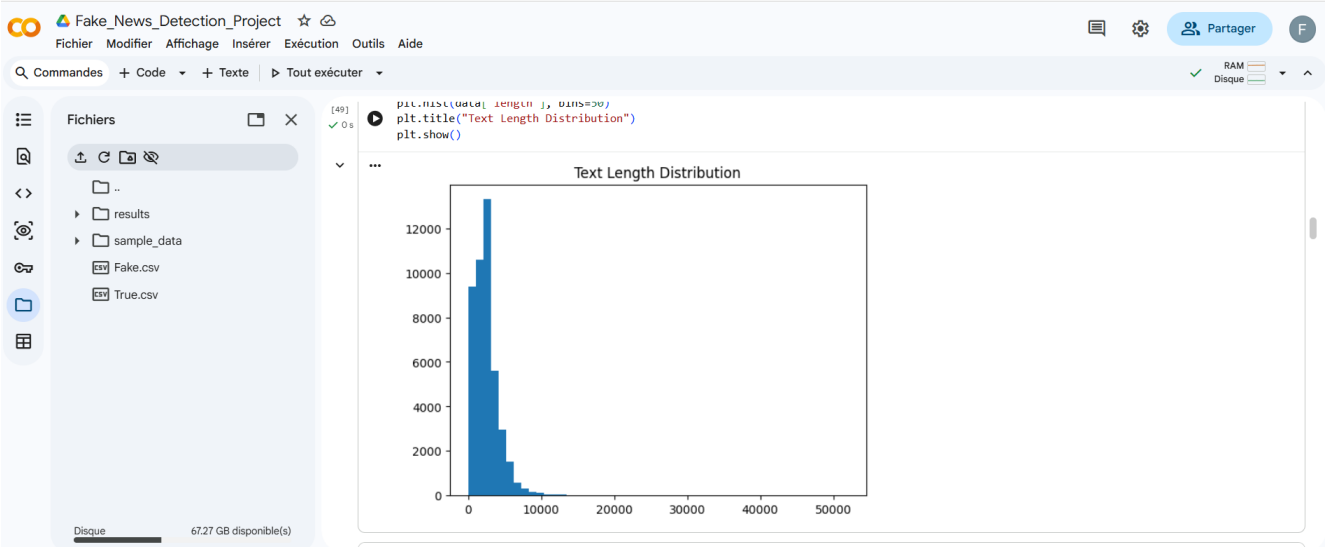
Train-test split configuration used before model building.

3. Exploratory Data Analysis (EDA)

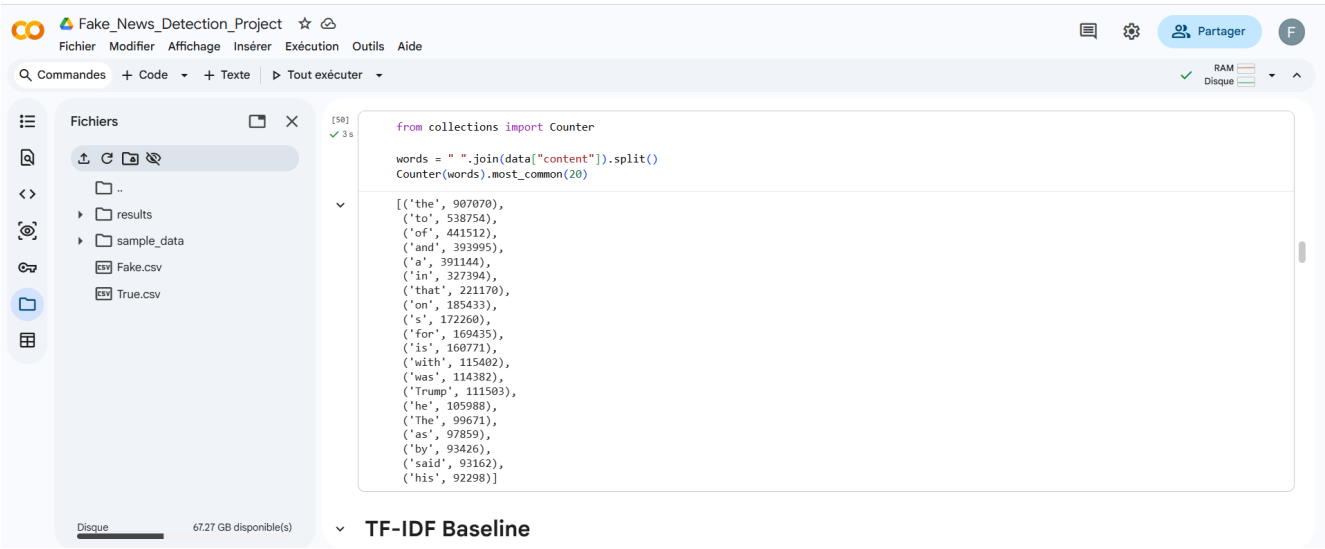
EDA is used to better understand the structure of the dataset. A class distribution chart shows that fake and real news samples are relatively balanced. A histogram of text lengths helps identify the typical size of the news articles, while a frequency analysis of common words highlights the dominant vocabulary and confirms the political nature of the dataset.



Class distribution between real and fake news.



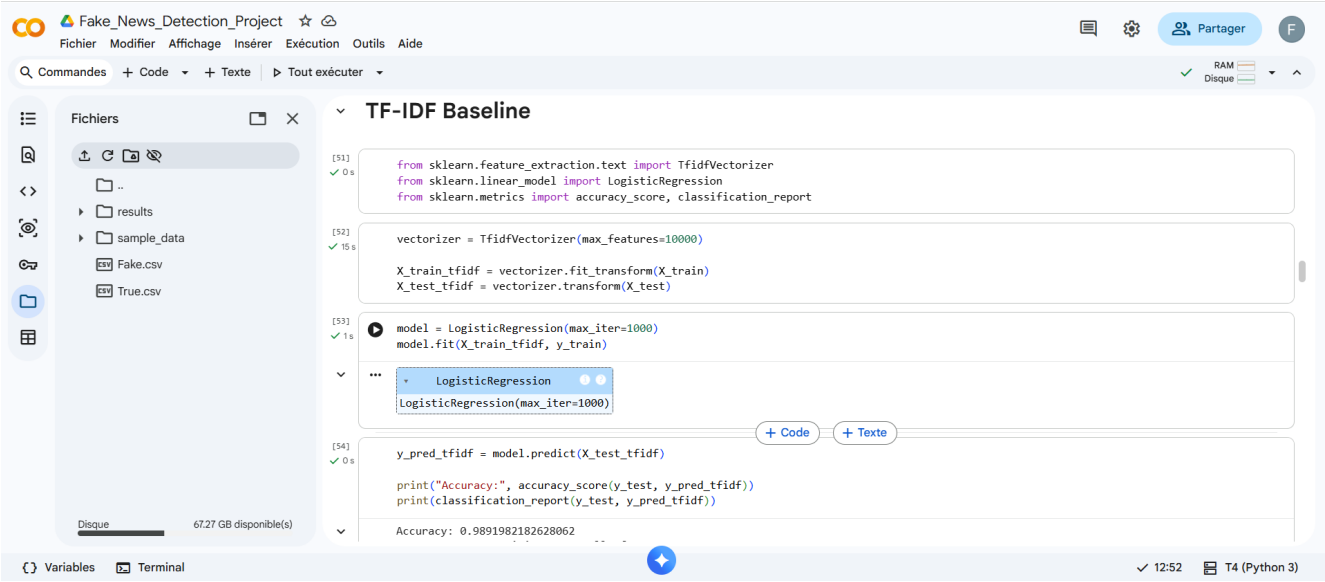
Text length distribution across the combined dataset.



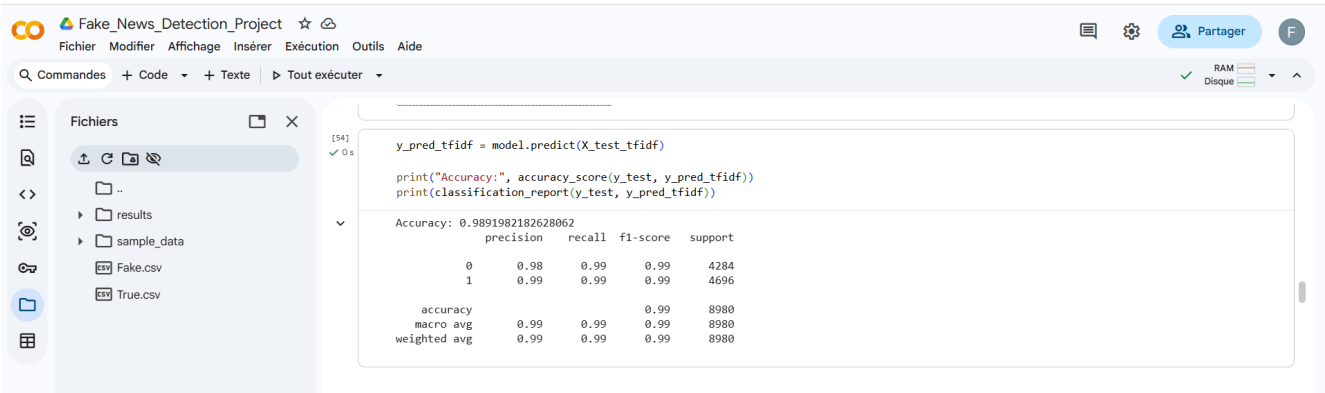
Most frequent words extracted from the corpus.

4. TF-IDF baseline model

As a traditional machine learning baseline, the project uses **TF-IDF vectorization** to transform text into numerical features, followed by **Logistic Regression** for classification. This baseline is important because it provides a strong reference point before applying a deep learning model.



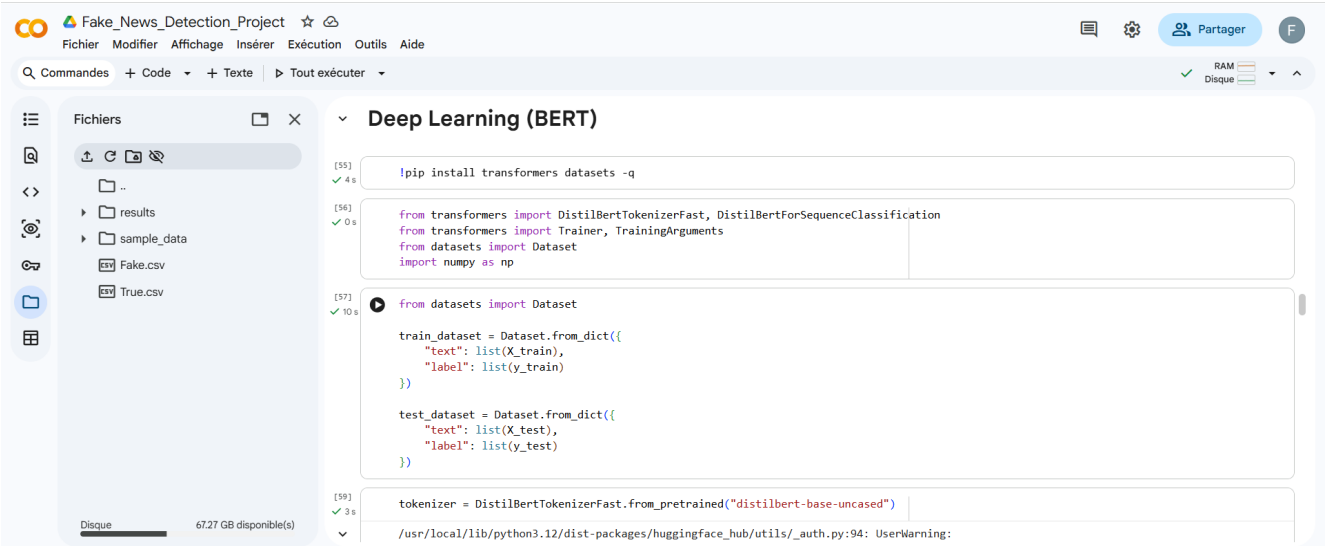
Implementation of the TF-IDF + Logistic Regression baseline.



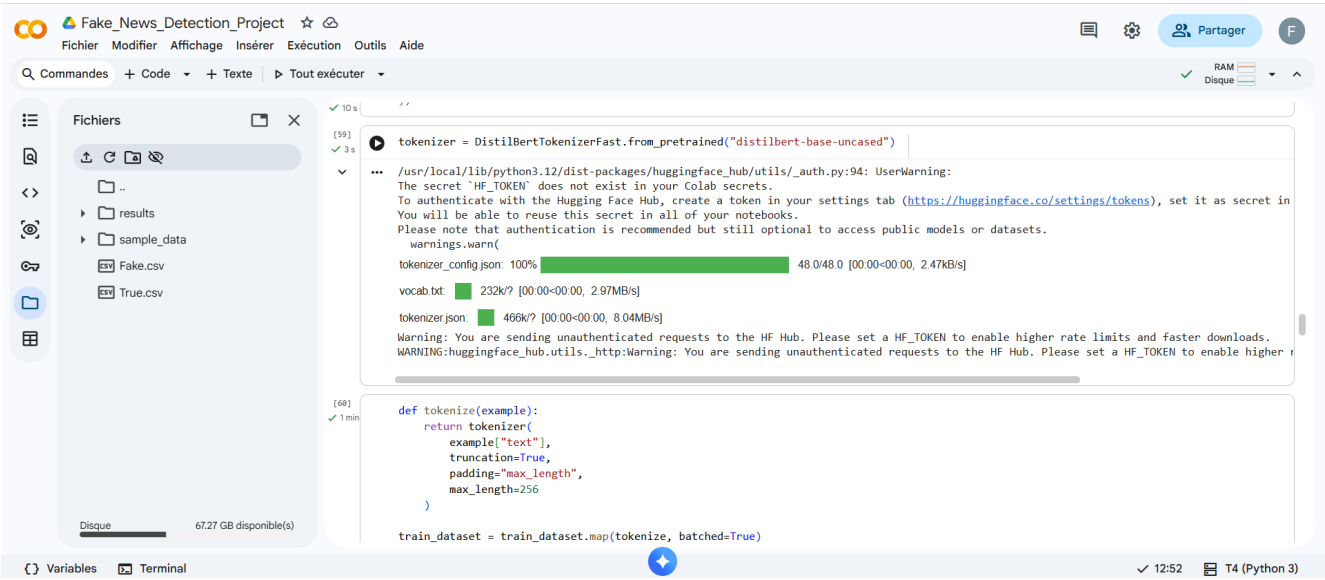
Baseline evaluation showing strong accuracy and classification metrics.

5. DistilBERT preparation and tokenization

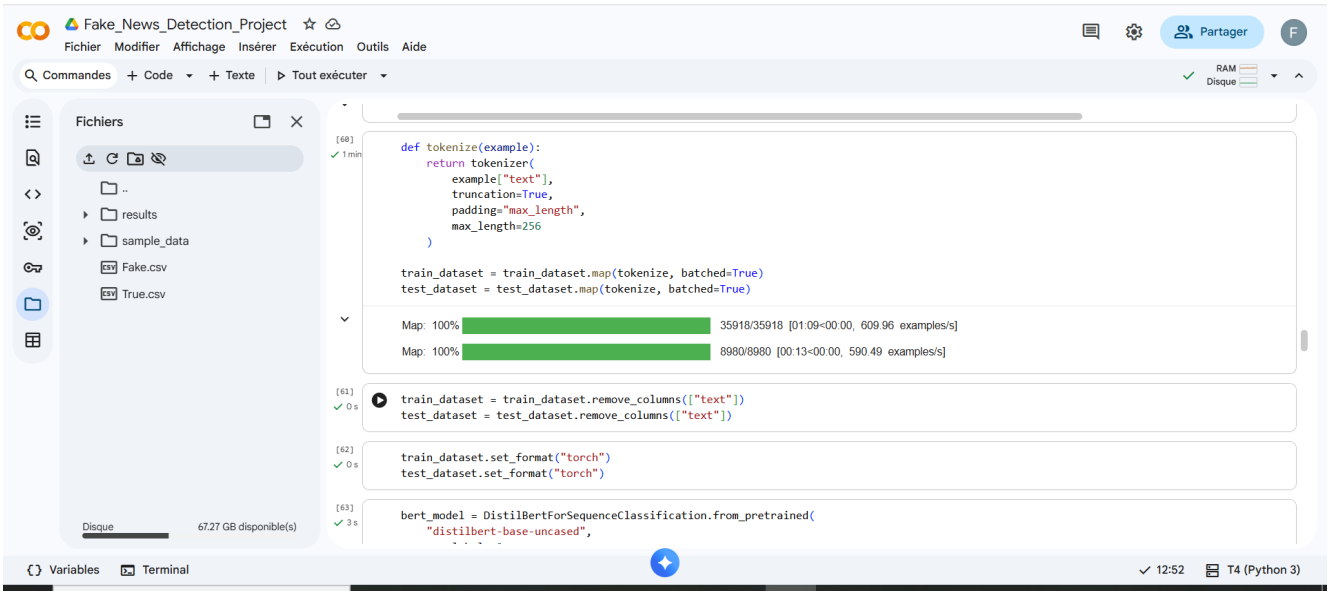
The deep learning part of the project uses **DistilBERT**, a compact Transformer model suitable for text classification on Google Colab. The training and testing datasets are converted into Hugging Face dataset objects, tokenized with the DistilBERT tokenizer, and formatted for PyTorch so they can be used by the Trainer API.



Import of Transformer libraries and creation of Hugging Face datasets.



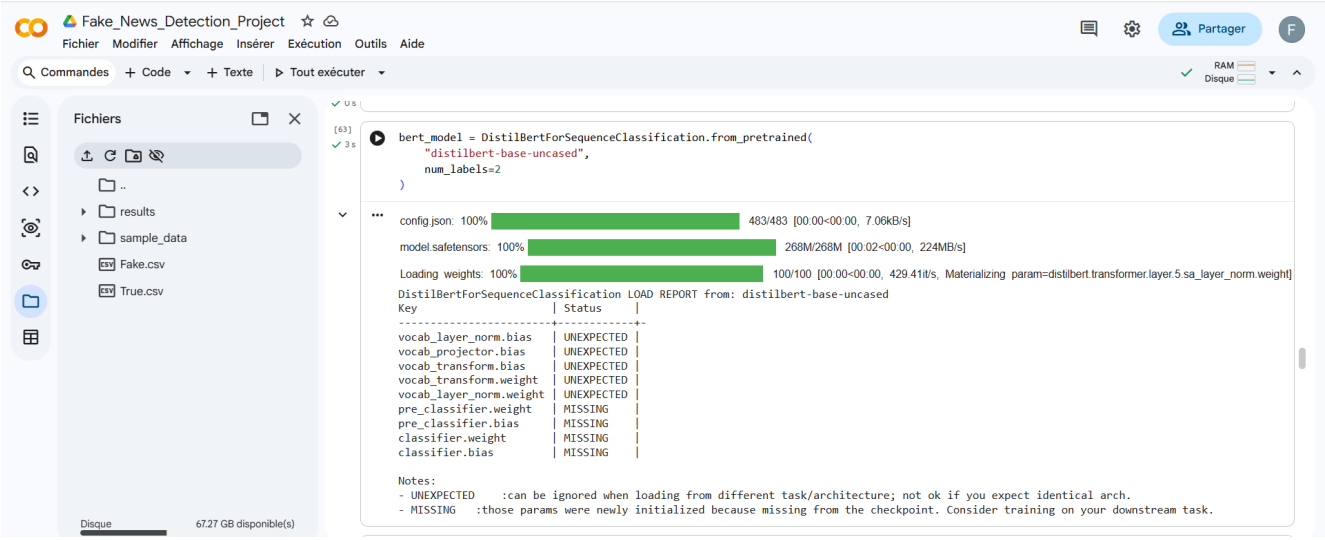
Tokenizer loading and tokenization setup.



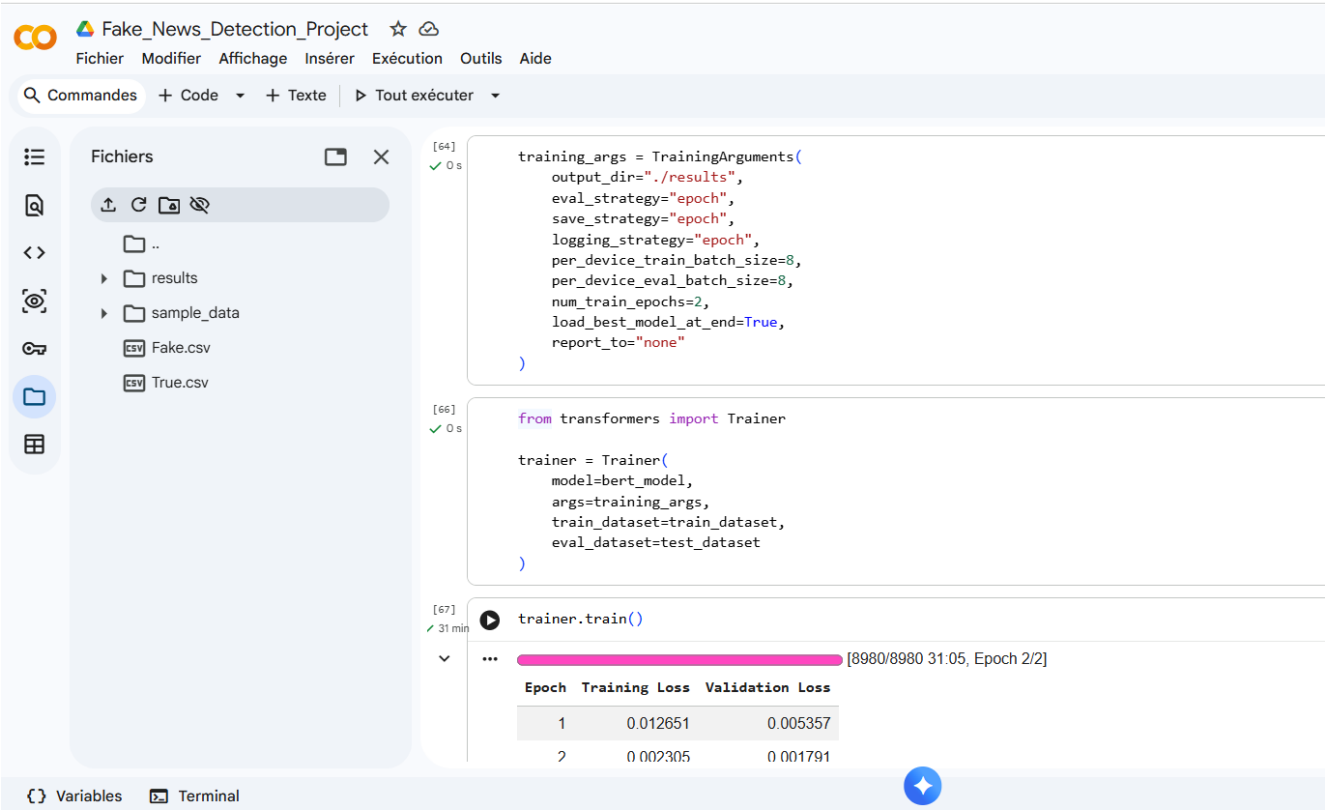
Tokenization completed, followed by removal of the raw text column and PyTorch formatting.

6. DistilBERT model configuration and training

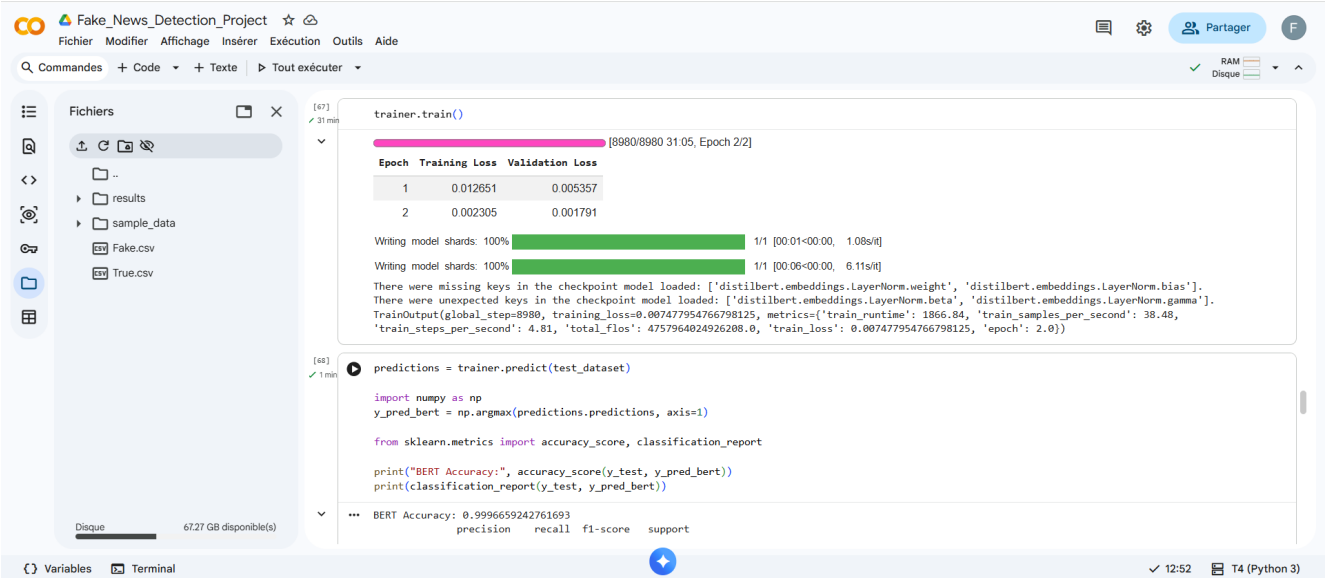
After tokenization, the DistilBERT classification model is loaded with two output labels. Training arguments are defined to control batch size, number of epochs, checkpointing, and evaluation at the end of each epoch. The model is then trained through the Hugging Face **Trainer** interface using GPU acceleration in Colab.



Loading the DistilBERT sequence classification model.



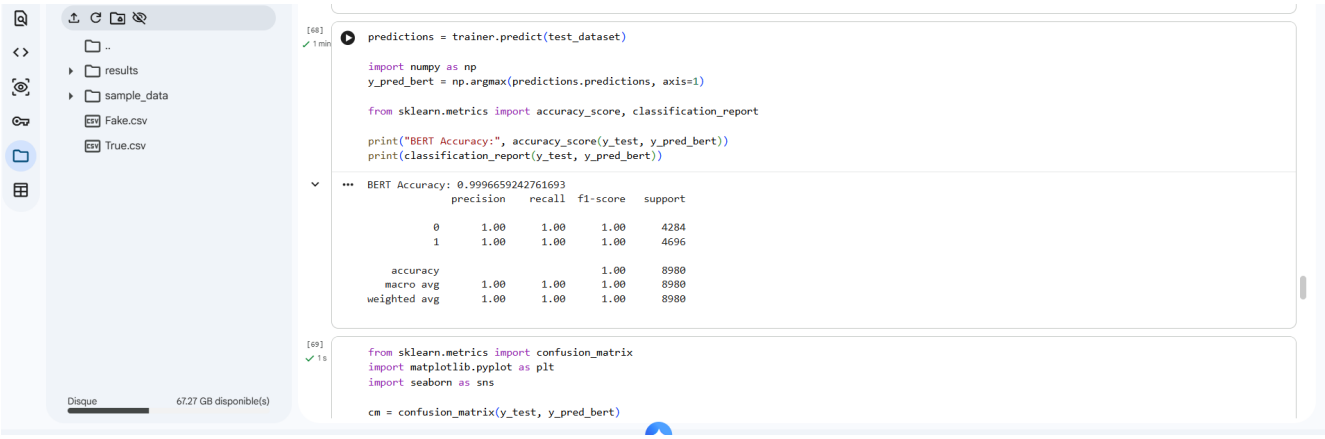
Definition of training arguments, Trainer object, and start of training.



End of training with training loss and validation loss across epochs.

7. BERT final evaluation

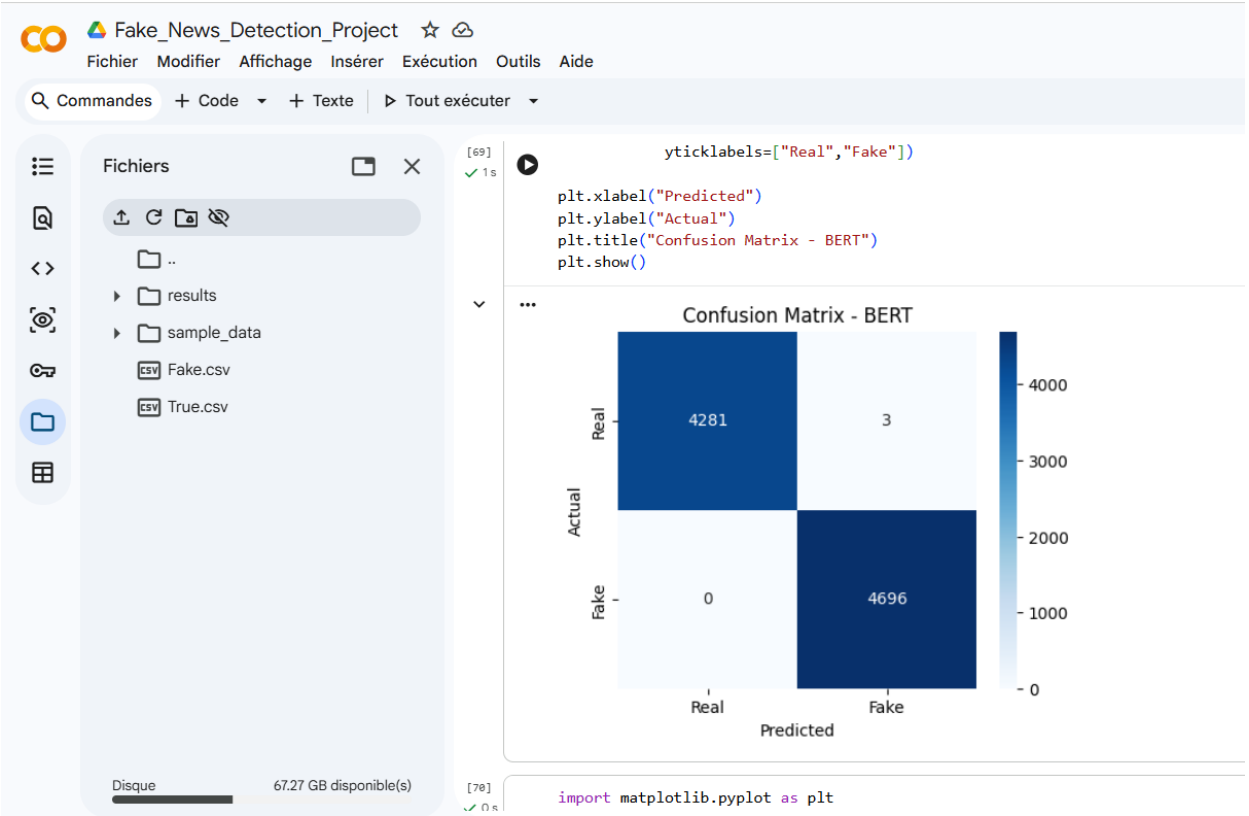
Once training is complete, predictions are generated on the test dataset. The final evaluation includes overall accuracy and the detailed classification report containing precision, recall, and F1-score. The BERT model achieves near-perfect results, indicating that the dataset is highly separable for this task.



Final evaluation of DistilBERT on the test set.

8. Confusion matrix analysis

The confusion matrix provides a clear view of the prediction errors made by the BERT model. In this project, the number of mistakes is extremely low. This confirms that the model is highly effective in distinguishing between fake and real news, especially for detecting fake samples.

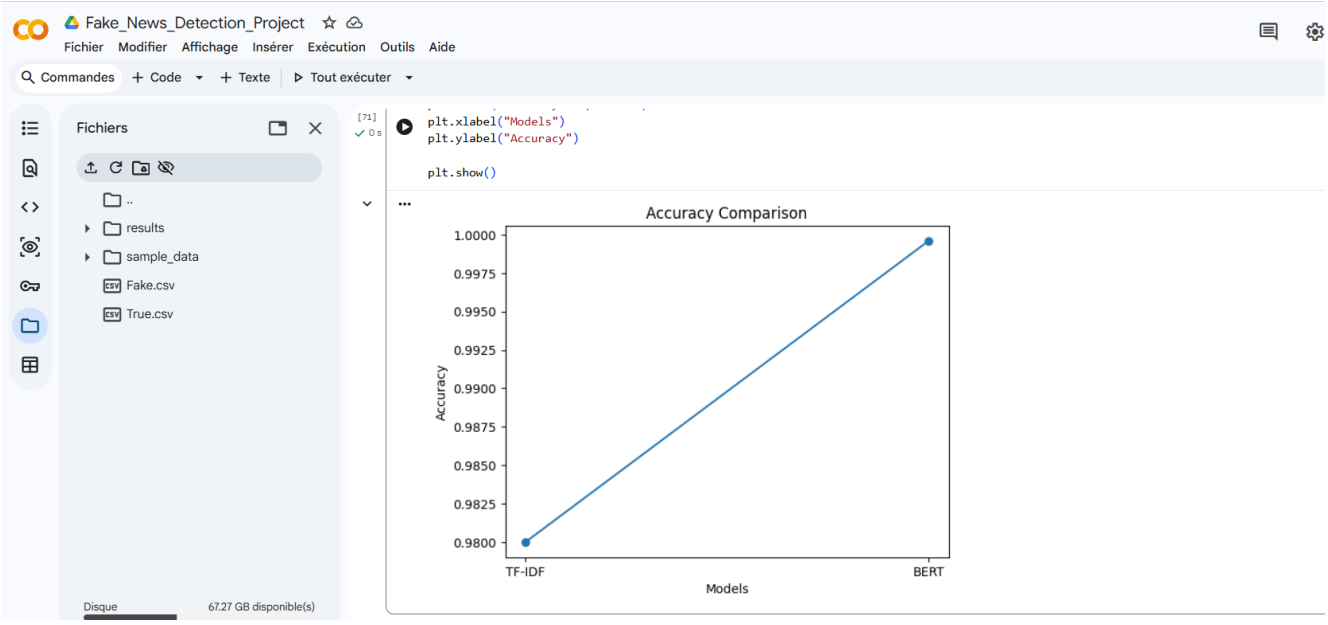


Confusion matrix of the DistilBERT model.

9. Model comparison

The final step compares the performance of the two approaches. The TF-IDF baseline already performs very well, but DistilBERT still surpasses it in accuracy. This demonstrates the advantage of contextual language models over classical bag-of-words-based approaches in fake news detection tasks.

Bar chart comparing TF-IDF and BERT accuracy.



Line chart showing the same model comparison trend.

10. Conclusion

This project successfully combines a traditional machine learning baseline and a deep learning Transformer model for fake news detection. The preprocessing and EDA stages helped validate the quality of the dataset, while the experimental comparison demonstrated that **DistilBERT** achieved the best overall performance. The report confirms that deep learning models with contextual understanding are especially effective for this NLP classification problem.

Summary of results:
- TF-IDF + Logistic Regression accuracy: approximately 0.9892
- DistilBERT accuracy: approximately 0.9997
- DistilBERT confusion matrix shows only a very small number of errors